searching for contiguous translations. On a TLB miss, a page table walk finds the desired translation. We aim to prevent any additional page walks when checking for contiguous entries adjacent to the requested translation. Since the page table walk accesses the last-level cache (LLC) and brings data in 64-byte cache lines, seven additional translations are fetched. These translations are brought without additional memory references; thus we check just them for contiguity. In practice, this approach restricts coalescing to a maximum of eight translations. Despite this restriction, CoLT eliminates a high number of TLB misses.

4.1.5. Replacement, invalidations, and attribute changes. CoLT-SA assumes standard LRU replacement policies. While there may be benefits in prioritizing entries with different coalescing amounts differently, we leave this for future work. We also assume a single set of attribute bits for all the coalesced entries, restricting coalescing opportunity. More sophisticated schemes supporting separate attribute bits per translation in a coalesced entry will improve our results. Furthermore on TLB invalidations, we flush out entire coalesced entries, losing information for pages that would be unaffected in standard TLBs. Gracefully uncoalescing TLB entries and only invalidating victim translations will perform even better. This too is the subject of future work.

4.1.6. Discussion of hardware overheads. To accommodate CoLT, the TLBs experience some key hardware changes. We argue, however, that these changes are modest. First, we believe that CoLT lookup remains low-overhead and does not impact TLB access cycle times. The initial tag match and check of valid bits is simple. The PPN generation logic addition is also low-overhead as the amount of coalescing is bounded (in our example, at best, an addition of four will be required). As such, readily-implementable combinational logic, similar to logic used to calculate prefetching strides and addresses or update branch predictor state, can calculate the physical page number. This is lower-overhead than prior prefetching schemes requiring dedicated adders [19].

Second, coalescing logic occurs on the TLB fill path rather than lookup, allowing subsequent TLB reads to proceed unimpaired. It is possible for subsequent reads to request translations that are part of the requested coalesced entries. These reads must wait until coalescing completes but we find these instances occur rarely. Furthermore, one might expect CoLT to require additional TLB ports to fill entries without conflicting with subsequent TLB reads. In our results, we assume no additional ports, finding that there is no significant performance degradation from this.

4.2. CoLT-FA Design and Implementation

Rather than supporting coalescing in set-associative TLBs and changing indexing schemes, we can instead coalesce into the fully-associative TLB (this structure is usually used exclusively for superpages). We refer to this as CoLT-Fully Associative (CoLT-FA).

4.2.1. Overall operation. The bottom half of Figure 5 delineates CoLT-FA operation. Assuming misses in all the TLBs (steps 1 and 2), a page walk is conducted in step 3. At this point, a cache line provides up to eight translations that can be checked for contiguity. Up to eight translations are now coalesced in step 4. If coalescable, the entry is loaded into the fully-associative TLB. If no coalescing is possible, it is loaded into the set-associative L1 and L2 TLBs.

On insertion into the fully-associative TLB, further coalescing is possible. Since contiguity may exist between the newly coalesced entry and a resident entry, the fully-associative TLB is scanned to seek further opportunities for coalescing. This scan is conducted while the requested TLB entry is returned to the processor. Further coalescing from the scan is done in step 5.

Empirically, we have found that due to the small size of the superpage-TLB, useful entries are frequently evicted. Therefore, for performance reasons, when bringing a coalesced entry into the fully-associative structure, we still bring just the requested entry (and not its coalesced neighbors) into the L2 TLB. While this does create some redundancy in terms of stored entries, we will show that performance is improved. Note that we leave the L1 TLB unaffected due to its much smaller capacity. Note also that CoLT-FA shares both superpage entries and coalesced entries in a single structure. One initial concern may be that if coalesced entries far outnumber superpage entries, the latter will be evicted from the fully-associative TLB. In practice, we find that this is not a problem for two reasons. First, superpages are used sparingly, requiring a very small number of entries in the buffer. Second, when used, these superpages are frequently accessed, meaning that they remain at the head of the LRU list, preventing their eviction.

4.2.2. Lookup operation. The top half of Figure 5 details CoLT-FA lookup. Each coalesced entry maintains a base virtual page number as the tag and a field that logs the number of entries coalesced. Unlike CoLT-SA, there are no coalescing restrictions due to indexing schemes. We find that using five bits for the coalescing length field suffices as this captures a contiguity of 32 pages. Each entry also stores the base physical page and attributes of all contiguous translations.

In step (a), *Range Checking Logic* compares the requested virtual page number against the range of translations stored by each entry of the fully-associative TLB. As shown, comparator and adder logic is required for the range check. If the virtual page is detected in the range, there is a TLB hit. At this point (step (b)), the *PPN Generation Logic* subtracts the tag base virtual page number from the requested virtual page number. This value is then added to the stored base physical page number to find the desired physical page.

4.2.3. Replacement, invalidations, and attribute changes. We assume standard LRU for the fully-associative structure. Due to its smaller size, we suspect though that smarter replacement policies will be even more effective. Furthermore, we share the same attribute bits for all coalesced entries and invalidate entire entries, but for larger amounts of coalescing. Despite this, we will show that CoLT-FA performs effectively.

4.2.4. Discussion of hardware overheads. Generally, CoLT-FA hardware for range checks and physical page number generation is more complex than CoLT-SA. To account for this, we reduce the size of the fully-associative TLB in CoLT-FA as compared to the baseline case without coalescing. Commercial systems tend to implement 16 to 24-entry fully-associative TLBs for superpages [17]. To ensure that the added lookup complexity does not bias our results, we assume only 8-entry fully-associative TLBs with coalescing. While a detailed circuit-level analysis of the lookup overhead is beyond the scope of this work, our decision to apply CoLT to a half-sized fully-associative TLB attempts to maintain the same access times.

An additional overhead arises from the secondary scan performed between existing fully-associative TLB entries and the coalesced entry being filled. While one might initially assume that we may need to increase port counts to ensure that subsequent lookups are not delayed, our implementation retains just the single port. Instead, we assume that the initial lookup of the fully-associative TLB iden-